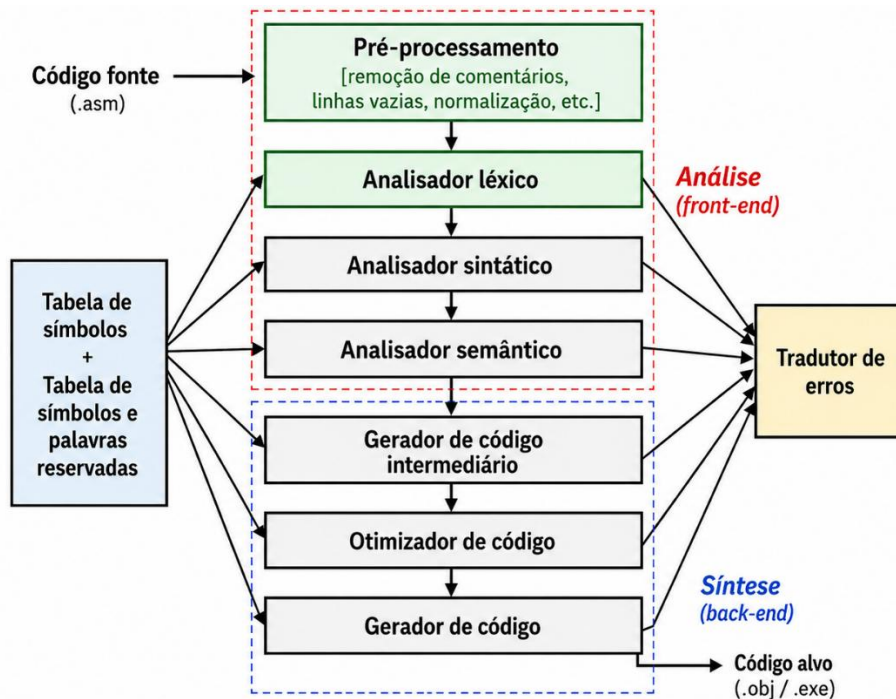


A ANÁLISE LÉXICA

1. Objetivo:

O objetivo deste trabalho é implementar, na linguagem C, um *analisador léxico para programas escritos em Assembly MIPS*, considerando o subconjunto de instruções e diretivas compatível com o simulador MARS especificado neste documento.



O analisador léxico deverá examinar o código-fonte caractere por caractere, agrupar os caracteres em lexemas e classificar cada lexema conforme os tokens definidos para a linguagem. Devem ser observadas as seguintes regras:

- O trabalho deverá ser desenvolvido pelo mesmo grupo formado para a fase de pré-processamento;
- Não será permitido utilizar ferramentas destinadas à geração automática de analisadores léxicos;
- A implementação deverá ser integralmente realizada em C;
- Trabalhos total ou parcialmente iguais receberão avaliação nula;
- Ultrapassados cinco dias da data definida para a entrega, nenhum trabalho será recebido.

O analisador léxico deverá:

- reconhecer os tokens do subconjunto da linguagem Assembly MIPS;
- identificar e registrar erros léxicos;
- manter uma Tabela de Símbolos;
- registrar a linha e a coluna inicial de cada token;
- gerar um arquivo de saída com extensão .lex, contendo os tokens reconhecidos;
- gerar um arquivo .ts, contendo a Tabela de Símbolos;
- gerar um arquivo .err, contendo os erros léxicos encontrados.

A implementação deverá ser baseada em um *Autômato Finito Determinístico (AFD)*.

Este trabalho está limitado à análise léxica. Portanto, não será responsabilidade do analisador verificar a quantidade de operandos de uma instrução, a compatibilidade entre registradores, a existência de um rótulo referenciado, a duplicidade de declaração de rótulos ou a validade sintática completa de uma instrução. Tais verificações pertencem às etapas sintática, semântica ou de montagem.

2. Funcionamento

O analisador léxico deverá ler um arquivo de saída do pré-processamento, contendo um programa escrito em Assembly MIPS e, a cada chamada, retornar o token correspondente ao próximo lexema reconhecido. Cada token deverá ser representado por uma estrutura contendo, pelo menos:

```
typedef struct {  
    char nome[50];  
    char lexema[100];  
    int linha;  
    int coluna;  
} Token;
```

Os campos deverão armazenar:

- nome: nome da categoria do token;
- lexema: sequência de caracteres encontrada no programa;
- linha: linha em que o lexema começa;
- coluna: coluna em que o lexema começa.

Espaços em branco, tabulações e quebras de linha não constituem tokens e deverão ser ignorados. As quebras de linha, entretanto, deverão atualizar corretamente os contadores de linha e coluna. No MARS, os comentários são iniciados pelo caractere #, continuam até o final da linha e devem ser eliminados na fase de pré-processamento.

Ao atingir o final do arquivo, o analisador deverá retornar um token especial denominado TK_EOF.

2.1. Tabela de Símbolos

A *Tabela de Símbolos (TS)* é uma estrutura utilizada para armazenar informações sobre os símbolos reconhecidos no programa. Em Assembly MIPS, ela pode conter instruções, diretivas, registradores e identificadores utilizados como rótulos. Neste trabalho, a Tabela de Símbolos deverá possuir as seguintes características:

- Inicialmente, deverá conter todas as instruções, diretivas e denominações de registradores previstas;
- Durante a análise, deverão ser acrescentados os identificadores utilizados como rótulos;
- Números, cadeias de caracteres e símbolos de pontuação não deverão ser armazenados;
- Um mesmo lexema não poderá ser inserido mais de uma vez;
- Por símbolo, deverão ser armazenados o lexema, sua categoria e a posição da primeira ocorrência;
- Instruções, diretivas e registradores deverão ser normalizados para letras minúsculas;

Atenção:

Para fins deste trabalho, os rótulos serão considerados sensíveis à diferença entre letras maiúsculas e minúsculas. Assim, início e Início representam símbolos diferentes.

Uma possível estrutura para uma entrada da tabela é:

```
typedef struct {  
    char lexema[100];  
    char categoria[30];  
    int primeiraLinha;  
    int primeiraColuna;  
} Simbolo;
```

Considere o seguinte fragmento:

```
.data  
mensagem: .asciiz "Resultado: "  
valor:    .word 10  
.text  
main:  
    lw $t0, valor
```

Parte da Tabela de Símbolos poderá ficar assim:

Lexema	Categoria	Primeira linha	Primeira coluna
.data	diretiva	1	1
.asciiz	diretiva	2	11
.word	diretiva	3	11
.text	diretiva	5	1
lw	instrução	7	5
\$t0	registrador	7	8
mensagem	identificador/rótulo	2	1
valor	identificador/rótulo	3	1
main	identificador/rótulo	6	1

As instruções, diretivas e denominações de registradores poderão constar na tabela desde sua inicialização. Nesse caso, a linha e a coluna poderão receber zero até que ocorra a primeira utilização no programa. Um rótulo é formado por um identificador seguido de dois-pontos. Por exemplo:

laco:

O analisador poderá retornar os tokens ID, para o lexema `laco`, e `SMB_COL`, para o caractere `:`. Ao encontrar essa construção, a Tabela de Símbolos deverá registrar `laco` como identificador utilizado como rótulo. As referências a rótulos também serão classificadas como ID:

j laco

Não será necessário verificar, durante a análise léxica, se o rótulo referenciado foi declarado. Além disso, referências antecipadas, isto é, realizadas antes da declaração do rótulo, deverão ser aceitas lexicalmente.

2.2. Erros léxicos

O analisador deverá detectar e registrar, no mínimo, os seguintes erros:

- caractere inválido;
- identificador malformado;
- número malformado;
- diretiva inválida ou malformada;
- registrador inválido ou malformado;
- cadeia de caracteres não fechada antes da quebra de linha;
- cadeia de caracteres não fechada antes do fim do arquivo;
- sequência de escape inválida em uma cadeia de caracteres.

Para cada erro, deverão ser informados tipo do erro; lexema ou caractere relacionado ao erro; linha; w coluna.

Considere o seguinte programa:

```
.text
main:
    li $t0, 10
    add $t1, $t0, @
    li $v0, 10
    syscall
```

Nesse caso, o analisador deverá registrar o caractere `@` como inválido:

<ERRO_CARACTERE_INVALIDO, @> 4 19

Também deverão ser considerados erros, por exemplo:

li \$t10, 5

O registrador `$t10` não pertence ao conjunto de registradores admitidos e deverá produzir um erro de registrador inválido.

.asciiz "Mensagem não encerrada

A cadeia iniciada por aspas não foi fechada antes da quebra de linha e deverá produzir um erro de cadeia não fechada.

li \$t0, 0xG1

O lexema 0xG1 não constitui um número hexadecimal válido e deverá produzir um erro de número malformado.

.dados

Como .dados não pertence ao conjunto de diretivas adotado neste trabalho, deverá ser registrada como diretiva inválida.

O arquivo .err deverá conter todos os erros encontrados, e não apenas o primeiro.

2.3. Tokens da linguagem

Um **token** é a categoria atribuída a um lexema reconhecido. No fragmento li \$t0, 10 o analisador deverá identificar:

Lexema	Token
li	INS_LI
\$t0	REG
,	SMB_COM
10	NUM_INT

Para tornar o trabalho compatível com o prazo e com os objetivos da disciplina, será adotado o subconjunto de Assembly MIPS definido a seguir.

Diretivas

As diretivas começam com um ponto e fornecem informações ao montador sobre segmentos, armazenamento e organização dos dados.

Diretiva	Token
.data	DIR_DATA
.text	DIR_TEXT
.word	DIR_WORD
.half	DIR_HALF
.byte	DIR_BYTE
.space	DIR_SPACE
.ascii	DIR_ASCII

.ascii	DIR_ASCII
.globl	DIR_GLOBL
.align	DIR_ALIGN

Qualquer lexema iniciado por ponto que não pertença a essa lista deverá ser registrado como diretiva inválida.

Instruções

As seguintes instruções e pseudoinstruções deverão ser reconhecidas:

Transferência de dados

Instrução	Token
li	INS_LI
la	INS_LA
move	INS_MOVE
lw	INS_LW
sw	INS_SW
lb	INS_LB
sb	INS_SB
lh	INS_LH
sh	INS_SH

Operações aritméticas

Instrução	Token
add	INS_ADD
addi	INS_ADDI
addu	INS_ADDU
sub	INS_SUB
subu	INS_SUBU
mul	INS_MUL
div	INS_DIV
mflo	INS_MFLO
mfhi	INS_MFHI

Operações lógicas, deslocamentos e comparação

Instrução	Token
and	INS_AND
or	INS_OR
xor	INS_XOR
nor	INS_NOR
sll	INS_SLL
srl	INS_SRL
slt	INS_SLT

Desvios e controle

Instrução	Token
beq	INS_BEQ
bne	INS_BNE
blt	INS_BLT
ble	INS_BLE
bgt	INS_BGT
bge	INS_BGE
j	INS_J
jal	INS_JAL
jr	INS_JR
syscall	INS_SYSCALL
nop	INS_NOP

As formas move, mul, blt, ble, bgt e bge podem ser implementadas internamente pelo MARS como pseudoinstruções. Para a análise léxica, entretanto, elas deverão ser reconhecidas como instruções válidas do subconjunto adotado.

Identificadores e rótulos

Os identificadores serão formados por uma letra ou sublinhado, seguidos de zero ou mais letras, dígitos ou sublinhados:

(letra | _) (letra | dígito | _)*

São exemplos válidos main, laco, soma_total, valor2 e _fim, diferentemente de 2valor, soma-total e inicio@, que são exemplos inválidos.

Os identificadores são usados principalmente como nomes de rótulos associados a endereços de instruções ou de dados:

```
mensagem: .asciiz "Olá"  
main:  
laco_1:
```

O identificador deverá gerar o token ID. O caractere `:` será reconhecido separadamente como `SMB_COL`.

Um identificador não poderá utilizar o mesmo lexema de uma instrução ou diretiva reservada. Assim, construções como `add:` ou `syscall:` não serão aceitas como declarações de rótulo.

Registadores

Os registradores deverão gerar o token genérico `REG`. Serão aceitos os seguintes nomes:

- `$zero`
- `$v0`, `$v1`
- `$a0`, `$a1`, `$a2`, `$a3`
- `$t0`, `$t1`, `$t2`, `$t3`, `$t4`, `$t5`, `$t6`, `$t7`, `$t8`, `$t9`
- `$s0`, `$s1`, `$s2`, `$s3`, `$s4`, `$s5`, `$s6`, `$s7`
- `$k0`, `$k1`
- `$gp`, `$sp`, `$fp`, `$ra`

Também poderão ser aceitas as formas numéricas:

- `$0`, `$1`, `$2`, ..., `$31`

Qualquer lexema iniciado por `$` que não corresponda a um desses registradores deverá produzir `ERRO_REGISTRADOR_INVALIDO`.

Exemplos:

Lexema	Resultado
<code>\$t0</code>	REG
<code>\$sp</code>	REG
<code>\$31</code>	REG
<code>\$t10</code>	erro
<code>\$32</code>	erro
<code>\$x1</code>	erro

Números inteiros

Os números inteiros deverão gerar o token `NUM_INT`. Serão aceitas as seguintes formas:

- decimal: 0, 15, 2025;
- decimal negativo: -1, -20, -32768;
- hexadecimal: 0x10, 0xFF, 0x10010000;
- hexadecimal negativo: -0x10.

Exemplos:

Lexema	Token
25	NUM_INT
-4	NUM_INT
0xFF	NUM_INT
-0x20	NUM_INT

Para simplificar a especificação, o sinal negativo será considerado parte do número quando estiver imediatamente seguido por um dígito ou pelo prefixo 0x. Não será necessário verificar, durante a análise léxica, se o valor numérico cabe na quantidade de bits exigida pela instrução. Essa verificação pertence a uma etapa posterior.

Números reais e instruções de ponto flutuante não fazem parte do subconjunto deste trabalho.

Cadeias de caracteres

As cadeias de caracteres serão delimitadas por aspas duplas e deverão gerar o token STRING:

```
.asciiz "Resultado: "  
.ascii "Microprocessadores"
```

As aspas farão parte do lexema registrado no arquivo .lex.

Ademais, deverão ser aceitas, no mínimo, as seguintes sequências de escape:

Sequência	Significado
\n	quebra de linha
\t	tabulação
\"	aspas duplas
\\	barra invertida
\0	caractere nulo

Uma cadeia não poderá continuar em outra linha. Se uma quebra de linha ou o final do arquivo for encontrado antes das aspas de fechamento, deverá ser registrado um erro.

Símbolos

Os seguintes símbolos deverão ser reconhecidos:

Símbolo	Token	Utilização
,	SMB_COM	separação de operandos ou valores
:	SMB_COL	declaração de rótulo
(	SMB_OPA	abertura de endereço baseado em registrador
)	SMB_CPA	fechamento de endereço baseado em registrador

Exemplo de endereçamento:

lw \$t0, 4(\$sp)

A saída correspondente deverá conter:

```
<INS_LW, lw>
<REG, $t0>
<SMB_COM, ,>
<NUM_INT, 4>
<SMB_OPA, (>
<REG, $sp>
<SMB_CPA, )>
```

O caractere # inicia um comentário e não gera token.

Resumo dos tokens

Os tokens do subconjunto podem ser agrupados da seguinte forma:

Categoria	Exemplos
Diretivas	DIR_DATA, DIR_TEXT, DIR_WORD, DIR_ASCII
Instruções	INS_LI, INS_LW, INS_ADD, INS_BEQ, INS_J
Identificadores	ID
Registradores	REG
Números	NUM_INT
Cadeias	STRING
Símbolos	SMB_COM, SMB_COL, SMB_OPA, SMB_CPA
Final de arquivo	TK_EOF
Erros	ERRO_CARACTERE_INVALIDO, ERRO_STRING_NAO_FECHADA, entre outros

3. Autômato Finito Determinístico

O AFD deverá representar os caminhos necessários para reconhecer, pelo menos:

- instruções e identificadores;
- diretivas iniciadas por ponto;
- registradores iniciados por \$;
- números decimais;
- números hexadecimais;
- números negativos;
- cadeias delimitadas por aspas;
- comentários iniciados por #;
- vírgula, dois-pontos e parênteses;
- caracteres inválidos.

Uma possível organização dos estados é estado inicial; estado de identificador ou instrução; estado de diretiva; estado iniciado por \$; estado de número decimal; estado de número hexadecimal; estado de número negativo; estado de cadeia; estado de sequência de escape; estado de comentário; estados finais para símbolos; e estado de erro.

Observações:

Quando um lexema formado por letras for reconhecido, o analisador deverá consultar a Tabela de Símbolos para decidir se ele representa uma instrução reservada ou um ID.

Da mesma forma, um lexema iniciado por ponto deverá ser comparado com o conjunto de diretivas, e um lexema iniciado por \$ deverá ser comparado com o conjunto de registradores válidos.

O estado de comentário deverá consumir todos os caracteres até encontrar uma quebra de linha ou o final do arquivo, sem produzir token.

4. Saída do analisador

O analisador deverá gerar três arquivos: .lex, .ts e .err.

4.1. Arquivo de tokens

O arquivo .lex deverá conter, para cada token:

<nome_token, lexema> linha coluna

Considere o código:

```
.text
main:
    li $v0, 10
    syscall
```

A saída poderá ser:

```
<DIR_TEXT, .text> 1 1
<ID, main> 2 1
<SMB_COL, :> 2 5
<INS_LI, li> 3 5
<REG, $v0> 3 8
<SMB_COM, ,> 3 11
<NUM_INT, 10> 3 13
<INS_SYSCALL, syscall> 4 5
<TK_EOF, EOF> 5 1
```

Os comentários, espaços, tabulações e quebras de linha não deverão aparecer nesse arquivo.

4.2. Arquivo da Tabela de Símbolos

O arquivo .ts deverá conter a Tabela de Símbolos completa, transcrita em notação CSV com atributos separados por vírgula. Uma possível organização é:

```
LEXEMA,CATEGORIA,LINHA,COLUNA
.data,diretiva,1,1
.text,diretiva,5,1
.asciiz,diretiva,2,11
lw,instrução,7,5
$t0,registrador,7,8
mensagem,identificador/rótulo,2,1
valor,identificador/rótulo,3,1
main,identificador/rótulo,6,1
```

Cada lexema deverá aparecer uma única vez.

4.3. Arquivo de erros

O arquivo .err deverá conter um registro para cada erro:

<tipo_erro, lexema> linha coluna

Exemplo:

```
<ERRO_REGISTRADOR_INVALIDO, $t10> 4 8  
<ERRO_CARACTERE_INVALIDO, @> 6 17  
<ERRO_STRING_NAO_FECHADA, "Mensagem"> 8 14
```

Caso nenhum erro seja encontrado, o arquivo poderá conter:

Nenhum erro léxico encontrado.

5. Entregáveis

O grupo deverá entregar:

Diagrama do AFD, elaborado no JFLAP ou, preferencialmente, com Graphviz, acessíveis a partir dos endereços JFLAP (<http://www.jflap.org/>) e Graphviz (<https://graphviz.org/download/>)

Código-fonte completo, contendo todos os arquivos .c e .h; arquivo principal; implementação do analisador léxico; conforme detalhado a seguir:

main.c	Com a função <code>main()</code> , responsável pelo recebimento e validação dos argumentos da linha de comando, pela abertura dos arquivos e pelo acionamento das funcionalidades do pré-processador;
preprocessador.c	Deverá conter a implementação das funções responsáveis pelas etapas de pré-processamento do código Assembly;
preprocessador.h	Deverá conter os protótipos das funções, constantes, tipos e demais declarações necessárias ao módulo de pré-processamento.
lexico.c	Deverá conter a implementação das funções responsáveis pela etapa de análise léxica do código Assembly;
lexico.h	Deverá conter os protótipos das funções, constantes, tipos e demais declarações necessárias ao módulo de análise léxica.

A análise léxica deverá ser realizada por meio da função `void AnaliseLexica(FILE *in, FILE *out)`, declarada no arquivo **lexico.h** e implementada no arquivo **lexico.c**. Essa função será responsável por receber, por meio do parâmetro *in*, o ponteiro para o arquivo que contém o código Assembly previamente normalizado e submetido ao pré-processamento e, por meio do parâmetro *out*, o ponteiro para o arquivo no qual serão gravados os tokens identificados. A função `main()`, após validar os argumentos fornecidos pela linha de comando e abrir adequadamente os arquivos de entrada e saída, deverá chamar obrigatoriamente `PreProcessamento(FILE in, FILE out)` e `AnaliseLexica(in, out)`. Caberá a essa função coordenar todas as operações previstas nessa etapa, como a leitura dos lexemas, sua classificação nas categorias léxicas da linguagem e o registro das respectivas linhas e colunas, além do reconhecimento de instruções, diretivas, registradores, identificadores, rótulos, números, strings e demais símbolos válidos. Essa organização reforça a separação entre o controle da execução, realizado em `main.c`, e a implementação das funcionalidades específicas do pré-processamento e analisador léxico, concentrada em `preprocessador.c` e `lexico.c`.

Relatório técnico, contendo descrição das estruturas de dados; descrição das funções implementadas; explicação do AFD; estratégia de controle de linha e coluna; estratégia de tratamento e recuperação de erros; explicação da Tabela de Símbolos; testes realizados; saídas geradas pelo analisador; e conteúdo final da Tabela de Símbolos.

Casos de teste, incluindo, no mínimo três programas lexicalmente corretos; três programas contendo erros léxicos; um teste com comentários; um teste com cadeia de caracteres; um teste com números decimais e hexadecimais; um teste com acesso à memória no formato deslocamento(registrador).

5.1. Execução

Considerando a organização do projeto nos arquivos `main.c`, `preprocessador.c`, `preprocessador.h`, `lexico.c` e `lexico.h`, a compilação poderá ser feita com o GCC da seguinte forma:

```
gcc main.c preprocessador.c analisador_lexico.c -o main.exe
```

Os arquivos `preprocessador.h` e `lexico.h` não precisam ser informados explicitamente no comando de compilação, pois serão incluídos nos arquivos `.c` correspondentes por meio das diretivas `#include "preprocessador.h"` e `#include "lexico.h"`. Após a compilação, o programa deverá ser executado por meio da linha de comando, recebendo obrigatoriamente três argumentos: o nome do arquivo Assembly de entrada, o nome do arquivo que receberá o resultado do pré-processamento e o nome do arquivo no qual será gravado o resultado da análise léxica.

A execução deverá seguir o formato:

```
./main.exe <arquivo_entrada.asm> <arquivo_saida.lex>
```

Por exemplo, em `./main.exe teste.asm teste.lex`

o programa deverá ler o código Assembly contido em `teste.asm`, realizar as operações de pré-processamento especificadas no trabalho e gravar o código normalizado em `teste.pre`. Em seguida, o arquivo normalizado deverá ser utilizado como entrada da análise léxica, cujo resultado será gravado em `teste.lex`.

O arquivo produzido pela análise léxica deverá apresentar os tokens identificados no código, contendo, para cada ocorrência, o lexema reconhecido, sua categoria léxica e sua localização no arquivo, indicada pela linha e pela coluna. Deverão ser reconhecidos e classificados, conforme as regras estabelecidas no trabalho, elementos como instruções, diretivas, registradores, identificadores, rótulos, números, strings, delimitadores e demais símbolos válidos da linguagem.

A função `main()` deverá coordenar essa sequência de execução, chamando inicialmente `PreProcessamento(in, out)` e, após a conclusão dessa etapa, `AnaliseLexica(in, out)`, com os arquivos adequados. A implementação específica de cada etapa deverá permanecer concentrada, respectivamente, em `preprocessador.c` e `lexico.c`.

Caso a quantidade de argumentos seja diferente da esperada ou não seja possível abrir algum dos arquivos, o programa deverá apresentar uma *mensagem de erro clara e encerrar sua execução de forma controlada*. Também deverão ser tratados os possíveis erros ocorridos durante a leitura ou a gravação dos arquivos.

5.2. Grade de correção

A avaliação do analisador léxico será realizada com base nos quesitos apresentados na grade de correção a seguir, os quais refletem os requisitos funcionais e de implementação estabelecidos para esta etapa do projeto. Cada quesito possui um peso percentual específico e será verificado, sempre que possível, por meio de testes automatizados, considerando aspectos como a estrutura e a compilação do código-fonte, a execução do programa, o tratamento de situações de erro e, principalmente, a correção das operações de pré-processamento.

A pontuação obtida em cada item será proporcional ao atendimento do respectivo requisito, sendo a nota final determinada pela soma dos percentuais alcançados nos diferentes quesitos avaliativos e posterior aplicação ao valor máximo estabelecido para esta etapa do projeto.

Nº	Quesito avaliativo	Pontuação máxima (%)	Pontuação atribuída (%)	Observações
1	Compilação, execução e organização modular	5%		
2	Integração com o pré-processamento	5%		
3	Implementação por meio de AFD	10%		
4	Reconhecimento e classificação dos tokens	30%		
5	Controle de linha, coluna e final do arquivo	5%		
6	Construção correta da tabela de Símbolos	20%		
7	Deteção e recuperação de erros léxicos	10%		
8	Geração de todos os arquivos de saída	5%		
9	Apresentação do relatório técnico	5%		
10	Apresentação de imagem com o AFD	5%		
	TOTAL	100%		